Relatório Técnico-Científico de Reestruturação Arquitetural do Sistema ERP JB Tintas

Autores: Equipe de Desenvolvimento – Projeto Integrador II (Bacharelado em Ciência de Dados / Tecnologia da Informação, UNIVESP)

Contexto: Refatoração de código-fonte e migração para padrões avançados de Engenharia de Software

1. Introdução e Contextualização

O presente relatório técnico-científico documenta o processo de refatoração estrutural aplicado ao código-fonte do Sistema Integrado de Gestão Empresarial (ERP) desenvolvido para o estabelecimento varejista JB Tintas. Na fase inicial de prototipagem, a aplicação empregava uma arquitetura monolítica tradicional baseada no ecossistema Django, na qual a camada de visualização acumulava regras de negócio, persistência de dados e interatividade em arquivos consolidados.

Embora funcional, tal abordagem apresentou limitações severas de manutenibilidade, acoplamento estrito e vulnerabilidade a conflitos de integração de código (*merge conflicts*) em ambiente de desenvolvimento colaborativo. Para mitigar tais óbices e atender rigorosos critérios de escalabilidade, testabilidade e separação de responsabilidades (*separation of concerns*), o sistema foi integralmente reestruturado sob os paradigmas arquiteturais **"Fat Models, Utility Services, Thin Views"** e a desvinculação completa entre a marcação estrutural (HTML) e o comportamento comportamental do cliente (JavaScript).

2. Fundamentação Arquitetural e Decisões de Projeto

A reestruturação orientou-se por princípios fundamentais da Engenharia de Software moderna, visando a modularização e o desacoplamento de componentes:

2.1. Camada de Apresentação (Thin Views e Extração de Front-End)

- **Estado Anterior:** As rotinas de manipulação do DOM (Document Object Model), validações assíncronas via `fetch` e tratamento de eventos encontravam-se acopladas aos templates HTML e centralizadas em um único arquivo de visualização de alto volume (`views.py`).
- **Modificação Científica:** As rotas foram fragmentadas em submódulos lógicos (`autenticacao_views.py`, `pdv_views.py`, `estoque_views.py`, `cliente_views.py`, `relatorios_views.py` e `config_views.py`). Adotou-se

o princípio das *Thin Views* (Vistas Finas), restringindo a responsabilidade da camada de visualização estritamente à recepção de requisições HTTP, validação de permissões de acesso baseadas em papéis (*Role-Based Access Control* - RBAC) e à delegação de fluxos para a camada de serviços.
- **Isolamento de Ativos (Assets):** Os scripts comportamentais foram extraídos dos templates e isolados em arquivos de script dedicados (`static/js/`), preservando a integridade dos diretórios nativos do framework e garantindo a modularidade do código cliente.

2.2. Camada de Domínio e Persistência (Fat Models)

- **Estado Anterior:** Os modelos relacionais concentravam-se exclusivamente na definição de esquemas de tabelas, configurando entidades anêmicas desprovidas de comportamento intrínseco.
- **Modificação Científica:** A base relacional foi segmentada por domínios de negócio (`produtos/`, `clientes/`, `usuarios/`, `vendas/`). Em conformidade com o padrão *Fat Models*, métodos computacionais e regras de integridade inerentes à própria entidade foram inseridos diretamente nas classes de modelo. Exemplificam essa mudança o método `tem_estoque_suficiente()` agregado à entidade de produtos e `calcular_comissao()` incorporada à entidade de colaboradores, encapsulando validações de estado e reduções de acoplamento externo.

2.3. Camada de Regras de Negócio (Utility Services)

- **Estado Anterior:** Processamentos transacionais complexos, tais como a baixa simultânea de estoque e a contabilização de programas de fidelidade, dividiam espaço com o roteamento HTTP.
- **Modificação Científica:** A lógica computacional e transacional foi isolada na camada de serviços (`services/`). Operações críticas que exigem integridade referencial e conformidade com as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade) foram dotadas do decorador transacional `@transaction.atomic`. Essa estratégia assegura que processos compostos — como o faturamento de PDV associado à dedução de inventário e distribuição de pontos de fidelidade — ocorram de maneira unificada, mitigando anomalias concorrentes e furos de inventário.

3. Metodologia de Implementação e Organização Estrutural

A transição arquitetural foi conduzida de maneira incremental, preservando a estabilidade do sistema legado (confinado em diretório de salvaguarda) e estabelecendo um rigoroso controle de dependências através do gerenciamento modular de pacotes Python (`__init__.py`).

A árvore de diretórios resultante reflete a padronização científica adotada:

1. `inventario/models/`: Centraliza os esquemas relacionais orientados por domínio, expostos globalmente via carregamento controlado no inicializador do pacote.
2. `inventario/services/`: Concentra os serviços utilitários transacionais e de processamento de dados externos (fidelidade, faturamento, métricas de dashboard).
3. `inventario/views/`: Agrupa os controladores HTTP de forma compartimentada, desprovidos de lógica computacional pesada.
4. `static/js/`: Armazena os scripts comportamentais desvinculados dos templates de apresentação.